

МОЛДАВСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Факультет Математики и Информатики

Департамент Информатики

По предмету Дискретная математика: теория графов.

Лабораторная работа № 2

**Отчет по программе: Поиск минимального доминирующего
множества**

Лектор: N. Cuciuc

Выполнил: D. Belov

Кишинев, 2026

1. Теория и определения

Доминирующее множество графа $G = (V, E)$ - это подмножество вершин $D \subseteq V$ такое, что каждая вершина из V , которая не входит в D , смежна хотя бы с одной вершиной из D .

Минимальное доминирующее множество - доминирующее множество, такое, что удаление любой вершины из множества D приводит к тому, что полученное множество перестаёт быть доминирующим множеством.

Задача поиска **минимального** доминирующего множества является NP-трудной (на данный момент не существует быстрого и эффективного способа решения). Для поиска доминирующего множества часто используют **жадный алгоритм**, который находит достаточно хорошее (но не всегда самое маленькое) решение за быстрое время.

В данной работе используется термин "**непокрытые**" вершины. Под этим термином будут пониматься вершины, которые одновременно не входят в доминирующее множество и не имеют ребро хотя бы с одной вершиной из доминирующего множества.

Термин "**эффективность покрытия вершины**" означает целочисленный показатель, который вычисляется по формуле: сама вершина + все смежные ей вершины, за исключением тех, которые не входят в множество непокрытых.

2. Описание алгоритма и принцип работы

- Создаётся копия набора всех вершин графа, которая служит индикатором непокрытых вершин. Инициализируется пустое множество для хранения результата.

- Программа входит в основной цикл, который активен до тех пор, пока множество непокрытых вершин не станет пустым.

- Для каждой вершины в графе вычисляется временный показатель эффективности. Этот показатель равен количеству смежных ей вершин, которые на данный момент еще не отмечены как покрытые + сама эта вершина, если она сама ещё не покрыта..

- Происходит сравнение и выбирается та вершина, у которой максимальная эффективность.

- Найденная вершина с максимальной эффективностью добавляется в доминирующее множество.

- Из общего набора непокрытых вершин исключаются: сама выбранная на шаге вершина и все вершины, которые смежны данной..

- Система проверяет, остались ли в графе вершины, которые не входят в доминирующее множество и не являются смежными ни с одним из его элементов. Если такие есть, происходит возврат к предыдущему шагам.

- После обнуления списка непокрытых вершин цикл прерывается. Программа выполняет коррекцию доминирующего множества (проверка на удовлетворение определению минимального доминирующего множества) и выводится результат.

3. Примеры применения DFS и работы программы

Размещение сервисных центров и узлов раздачи

В логистике алгоритм используется для выбора мест расположения объектов (складов, базовых станций, постов экстренных служб). Решение задачи гарантирует, что каждый потребитель (вершина графа) будет находиться в зоне покрытия как минимум одного сервисного центра при минимальных общих затратах на установку оборудования.

Пример работы программы

Входные данные:

3 - 2
3 - 1
3 - 5
2 - 5
2 - 4
4 - 5
1 - 6
1 - 7
6 - 7

Программа строит структуру (список смежности):

3: 2, 1, 5
2: 3, 5, 4
1: 3, 6, 7
5: 3, 2, 4
4: 2, 5
6: 1, 7
7: 1, 6

Множество непокрытых вершин содержит все вершины.

Доминирующее множество не содержит вершин.

Для каждой вершины вычисляется эффективность добавления вершины на текущей итерации:

3: 2, 1, 5	эф.: 1 (сама вершина) + 3 (смежные) = 4
2: 3, 5, 4	эф.: 4
1: 3, 6, 7	эф.: 4
5: 3, 2, 4	эф.: 4

4: 2, 5	эф.: 3
6: 1, 7	эф.: 3
7: 1, 6	эф.: 3

Так как у нескольких вершин одинаковый показатель эффективности, выбирается та, которая встречается в структуре первой (ввод влияет на результат). Вершина 3, а также все ей смежные вершины удаляются из множества непокрытых. Непокрытые: 4, 6, 7. В доминирующее множество записывается 3.

Подсчитывается новая эффективность добавления для каждой из вершины графа:

3: 2, 1, 5	эф.: 0 (нет совпадений с множеством непокрытых)
2: 3, 5, 4	эф.: 1 (вершина 4)
1: 3, 6, 7	эф.: 2
5: 3, 2, 4	эф.: 1
4: 2, 5	эф.: 0
6: 1, 7	эф.: 2
7: 1, 6	эф.: 2

Выбирается вершина 1, так как она имеет наибольшую эффективность и встречается первой.

Из множества непокрытых исключаются 6 и 7.

Вершина 1 добавляется в доминирующее множество.

Подсчитывается новая эффективность.

3: 2, 1, 5	эф.: 0
2: 3, 5, 4	эф.: 1
1: 3, 6, 7	эф.: 0
5: 3, 2, 4	эф.: 1
4: 2, 5	эф.: 0
6: 1, 7	эф.: 0
7: 1, 6	эф.: 0

Так как 2 встречается первой, то она добавляется в доминирующее множество. Множество непокрытых вершин пустое, что является условием выхода из основного цикла.

Предполагалось, что последовательный подход создаст доминирующее множество, ведь выход из цикла совпадает с получением доминирующего множества. Однако при таком подходе есть шанс, что одна из вершин будет смежна только другим вершинам из доминирующего множества или вершинам, которые они покрывают. простыми словами нарушается само определение минимального доминирующего множества: минимальное доминирующее множество - доминирующее множество, убрав любую вершину из которого, перестанет быть доминирующим. Так как определение звучит как проверка, то в программе она получила именно такую реализацию. После того как мы получили сырой результат (3, 1, 2), выполняется дополнительная проверка на удовлетворение определению. Из множества

исключаются все вершины, которые можно исключить. В результате у нас получается минимальное доминирующее множество.

```
Введите ребра (например, 1 2). 'stop' для выхода:
> 3 2
> 3 1
> 3 5
> 2 5
> 2 4
> 4 5
> 1 6
> 1 7
> 6 7
> stop

--- РЕЗУЛЬТАТ ---
Доминирующее множество: 2, 1
PS C:\Users\PARARADOX\AppData\Local\Programs\Microsoft VS Code> █
```

4. Выводы и оценка сложности

Временная сложность:

В худшем случае алгоритм делает $O(V(V + E))$ операций, где V - количество вершин, а E - ребер. Для каждой добавленной в ответ вершины мы заново сканируем граф для поиска лучшего кандидата.

Пространственная сложность:

Список смежности: $O(V + E)$

Преимущества текущей реализации:

1. Использование динамических структур данных (dict, set) в Python позволяет работать с графами любого размера.
2. Устойчивость к вводу дубликатов ребер.
3. Автоматическая обработка изолированных вершин.
4. Нет никаких ограничений на названия вершин, однако из этого следует отсутствие сортировки при вводе.

5. Листинг программы

```
def minimize_dominating_set(graph, dominating_set):
```

```
    """
```

```
    Удаляет лишние вершины из доминирующего множества,
    делая его минимальным по включению.
```

```

"""
minimal_set = list(dominating_set)
all_nodes = set(graph.keys())

for node in list(minimal_set):
    minimal_set.remove(node)

    current_covered = set()
    for active_node in minimal_set:
        current_covered.add(active_node)
        current_covered.update(graph[active_node])

    if current_covered != all_nodes:
        minimal_set.append(node)

return set(minimal_set)

def find_dominating_set_greedy(graph):
    uncovered = set(graph.keys())
    dominating_set = set()

    while uncovered:
        best_node = None
        max_newly_covered = -1

        for node in graph:
            newly_covered = (graph[node] | {node}) & uncovered
            if len(newly_covered) > max_newly_covered:
                max_newly_covered = len(newly_covered)
                best_node = node

        if best_node is None: break

        dominating_set.add(best_node)
        uncovered -= (graph[best_node] | {best_node})

    return dominating_set

graph = {}
print("Введите ребра (например, 1 2). 'stop' для выхода:")

while True:
    try:
        line = input("> ").strip()
        if line.lower() == 'stop': break
        if not line: continue

        parts = line.split()

```

```

if len(parts) != 2:
    print("Пропуск некорректной строки")
    continue

u, v = parts
if u not in graph: graph[u] = set()
if v not in graph: graph[v] = set()

if u != v:
    graph[u].add(v)
    graph[v].add(u)

except EOFError:
    break

if not graph:
    print("\nГраф пуст.")
else:
    # 1. Получаем жадное множество
    greedy_res = find_dominating_set_greedy(graph)

    # 2. Минимизируем его (убираем лишние вершины)
    final_res = minimize_dominating_set(graph, greedy_res)

    print("\n--- РЕЗУЛЬТАТ ---")
    res_nodes = ", ".join(final_res)
    print(f"Доминирующее множество: {res_nodes}")

```